# Cypher 5 vs Cypher 25 — Limites, apports et plan de projet

## 1. Contexte et objectif

Ce document synthétise le travail conceptuel réalisé jusqu'à présent. L'objectif est de comprendre les limites de Cypher 5 pour l'écriture de requêtes expressives sur des chemins de graphes, d'expliquer les apports de Cypher 25, et de montrer comment cette comparaison sera appliquée à un jeu de données réel (NYCFLIGHT13) à l'aide de Neo4j et PostgreSQL.

## 2. Limites de Cypher 5

Cypher 5 est efficace pour le filtrage local de motifs de graphes, mais il manque d'expressivité native pour exprimer des contraintes globales sur des chemins (ordre, unicité, agrégation). Pour contourner ces limites, Cypher 5 introduit des constructions de traitement de listes telles que nodes(p), relationships(p), reduce, all, any, none et size.

Ces constructions permettent de rendre certaines requêtes exprimables, mais elles entraînent de graves problèmes de performance lorsqu'elles sont utilisées pour filtrer des chemins. Le moteur doit générer l'ensemble des chemins candidats, construire les listes correspondantes, puis appliquer les contraintes dans les clauses WHERE. Cette stratégie conduit à une explosion combinatoire, à des temps d'exécution imprévisibles et à des timeouts, même sur des graphes de petite taille.

Schémas problématiques typiques en Cypher 5 :

1. Filtrage de chemins à l'aide de reduce ou de prédicats sur des listes dans WHERE.
2. Contraintes globales sur les chemins (valeurs strictement croissantes, unicité).
3. Simulation d'automates ou d'expressions régulières complexes via des listes.

## 3. Apports de Cypher 25

Cypher 25 introduit de nouvelles constructions pour les requêtes de chemins expressives, notamment les motifs de chemins quantifiés (quantified path patterns) et des mécanismes de réduction contrôlés. Ces fonctionnalités permettent d'exprimer les contraintes directement dans le motif de graphe.

Contrairement à Cypher 5, Cypher 25 applique les contraintes pendant l'exploration du graphe. Les chemins invalides sont éliminés au fur et à mesure de leur construction, ce qui réduit considérablement l'espace de recherche, le temps d'exécution et la consommation mémoire.

Principaux avantages de Cypher 25 :

1. Contraintes intégrées directement aux motifs de chemins.
2. Élagage (pruning) précoce des chemins invalides.
3. Réduction du recours aux listes dans les clauses WHERE.
4. Plans d'exécution plus optimisables et plus prévisibles.

## 4. Jeu de données : NYCFLIGHT13

Le projet s'appuie sur le jeu de données NYCFLIGHT13 issu de l'écosystème R. Ce jeu de données décrit l'ensemble des vols au départ de New York en 2013, incluant les compagnies aériennes, les aéroports, les

horaires, les distances et les retards.

Une compréhension précise du modèle de données est indispensable pour concevoir des requêtes pertinentes. Les entités clés sont notamment les aéroports, les vols et les compagnies, avec des propriétés temporelles et numériques exploitables pour les requêtes de chemins.

## 5. Méthodologie et étapes du projet

Le projet sera mené selon les étapes suivantes :

1. Identifier des requêtes de chemins complexes sur le graphe des vols (par exemple, itinéraires temporellement cohérents ou sans répétition d'aéroports).
2. Implémenter ces requêtes en Cypher 5 en utilisant des constructions de listes (nodes, relationships, reduce, all, size).
3. Réécrire les mêmes requêtes logiques en Cypher 25 à l'aide des motifs de chemins quantifiés et des contraintes intégrées.
4. Implémenter des requêtes équivalentes en SQL récursif avec PostgreSQL (WITH RECURSIVE).
5. Comparer les comportements d'exécution, les plans, les performances et la scalabilité entre Cypher 5, Cypher 25 et PostgreSQL.

## 6. Résultat attendu

Le résultat attendu est une démonstration claire, à la fois conceptuelle et expérimentale, montrant que si Cypher 5 permet d'exprimer des requêtes complexes sur les chemins grâce au traitement de listes, cette approche est coûteuse et peu scalable. Cypher 25 permet d'atteindre le même niveau d'expressivité tout en maîtrisant le coût d'exécution, en intégrant les contraintes directement dans les motifs de graphes.